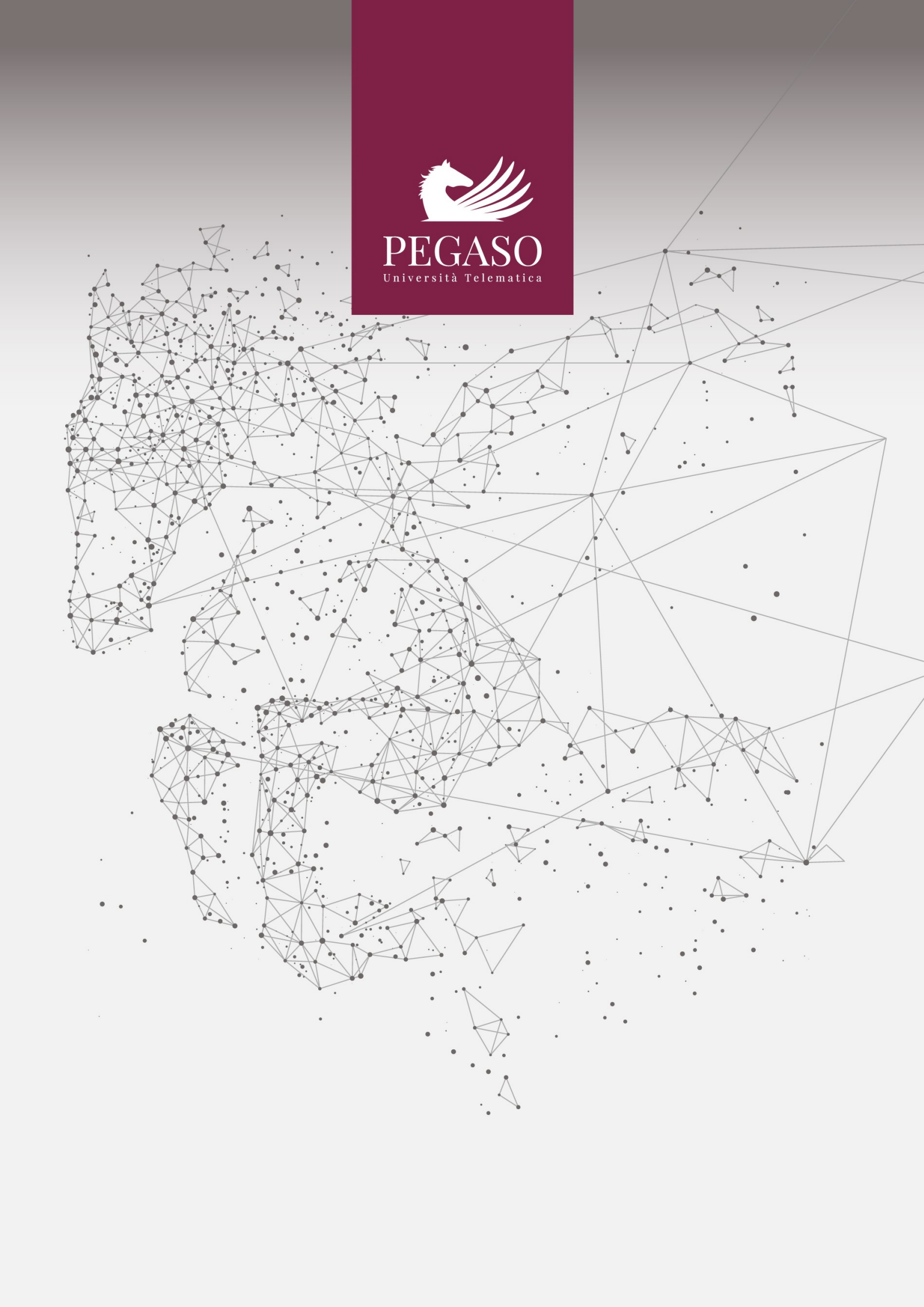

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. VISUALIZZAZIONE DEI GRAFI CON NETWORKX E MATPLOTLIB 4
3. VISUALIZZAZIONE DEI GRAFI CON GEPHI 12
4. ANALISI DEI GRAFI 14
BIBLIOGRAFIA 24

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano due modalità principali per la visualizzazione dei grafi. In particolare, la prima modalità è relativa all'uso combinato di Networkx e Matplotlib, mentre la seconda modalità è relativa a Gephi, un software esterno a Python per la visualizzazione e l'esplorazione di tutti i tipi di grafici e reti. Inoltre, vengono illustrate le funzionalità offerte da Networkx per l'analisi dei grafi per l'estrazione delle componenti connesse, il calcolo del coefficiente di clustering, il calcolo delle centralità, il calcolo della modularità, l'individuazione delle cricche e delle comunità di rete.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Visualizzazione dei grafi con Networkx e Matplotlib

Come è semplice osservare, le funzioni di stampa dei nodi e degli archi non sono sufficienti per una corretta e veloce comprensione della **struttura di un grafo.** NetworkX fornisce solo *funzionalità di base per la visualizzazione di grafi.* L'***obiettivo principale*** di Networkx è, infatti, quello di permettere l'**analisi del grafo** piuttosto che permetterne la visualizzazione. Per la visualizzazione di base dei grafi, Networkx utilizza la libreria Matplotlib. Per la visualizzazione di un grafo è necessario selezionare il **layout.** Tra i più importanti layout disponibili ci sono:

- circular;
- planar;
- shell;
- spectral.

Il formato tipico della funzione sarà **draw_tipolayout**() (es. *draw_circular*(), *draw_planar*(), ecc.). Alla funzione **draw_tipolayout**() verranno passati alcuni argomenti tra i quali:

- grafo da visualizzare;
- parametri *opzionali* tra cui:
  - *with_labels*=True o False permette, rispettivamente, di visualizzare o non visualizzare anche l'etichetta del nodo;
  - *node_size* e *node_color* che permettono di specificare la grandezza e il colore dei nodi (può essere anche un vettore);
  - *nodelist* e *edgelist* che specificano, rispettivamente, quali nodi e quali archi visualizzare (di default vengono visualizzati tutti i nodi e tutti gli archi).

Procediamo alla **generazione** di un grafo molto semplice con il codice seguente.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
%matplotlib inline
```

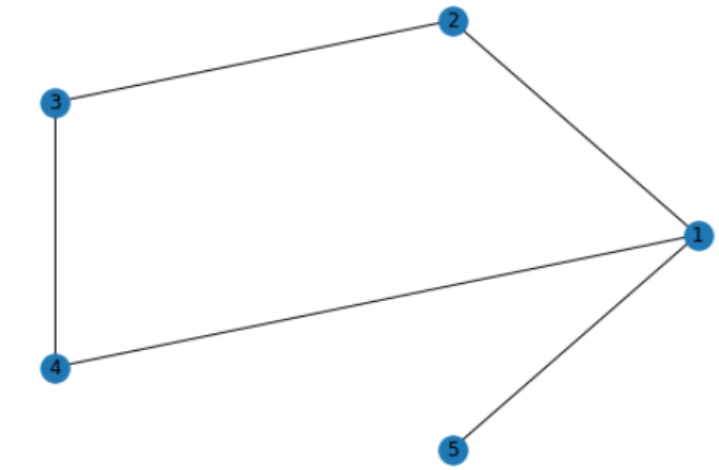
```
import networkx as nx
import matplotlib.pyplot as plt

g = nx.Graph()

g.add_edge(1, 2)
g.add_edge(2, 3)
g.add_edge(3, 4)
g.add_edge(1, 4)
g.add_edge(1, 5)
```

Successivamente, visualizziamo tale grafo secondo diversi layout offerti da Networkx-Matplotlib. La visualizzazione del grafo (incluse le etichette dei nodi) con layout **circular** è la seguente:

```
nx.draw_circular(g, with_labels = True)
plt.show()
```

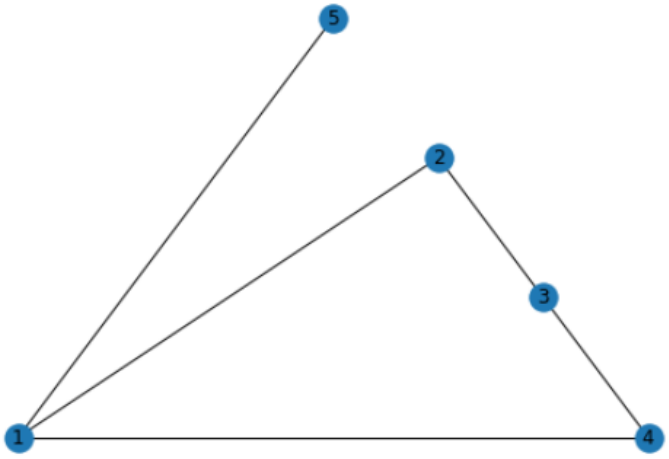


PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

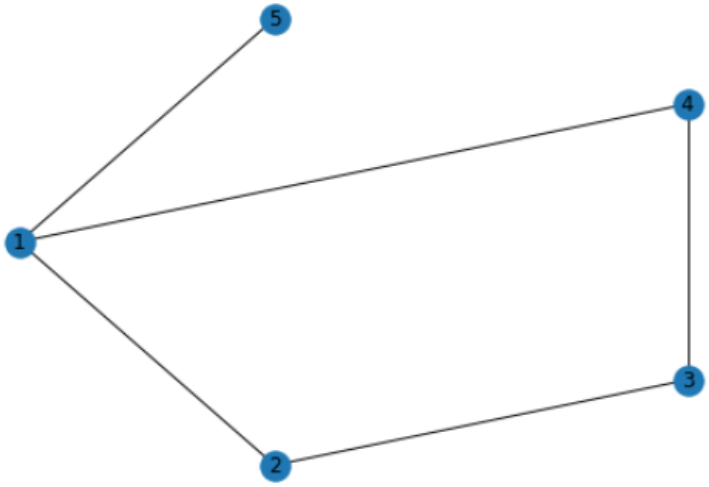
La visualizzazione del grafo (incluse le etichette dei nodi) con layout **planar** è la seguente:

```
plt.clf()  # azzera il plot precedente
nx.draw_planar(g, with_labels = True)
plt.show()
```



La visualizzazione del grafo (incluse le etichette dei nodi) con layout **shell** è la seguente:

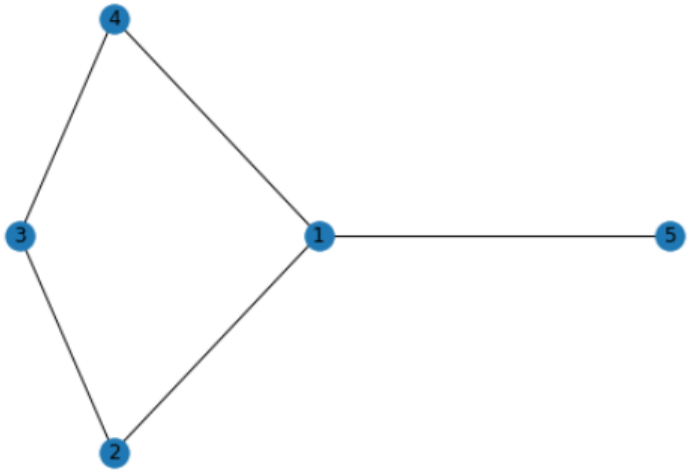
```
plt.clf()
nx.draw_shell(g, with_labels = True)
plt.show()
```



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La visualizzazione del grafo (incluse le etichette dei nodi) con layout **spectral** (e salvataggio dell'immagine su un file png) è la seguente:

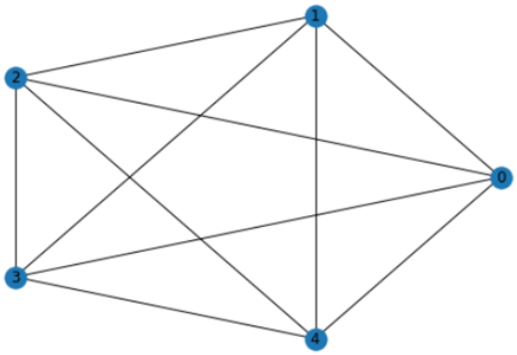
```
plt.clf()
nx.draw_spectral(g, with_labels = True)
plt.savefig("grafo.png")
```

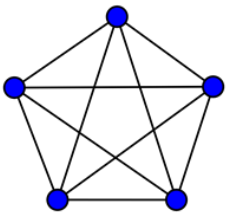


Visualizziamo, ora, alcuni grafici classici come il **grafo completo**. La funzione per ottenerlo è **complete_graph**() a cui va passato come argomento il numero dei nodi del grafo completo (nella parte destra della figura seguente si riporta il modello corrispondente di grafo desiderato).

```
import networkx as nx
import matplotlib.pyplot as plt

plt.clf()
K_5 = nx.complete_graph(5)
nx.draw_circular(K_5, with_labels = True)
plt.show()
```

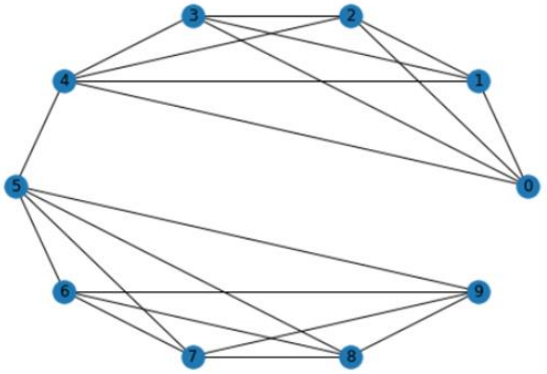


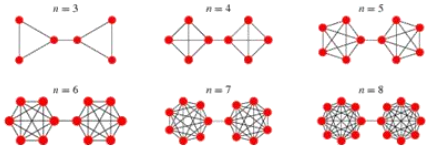


Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Visualizziamo, ora, alcuni grafici classici come il **grafo Barbell**. La funzione per ottenerlo è **barbell_graph**() (nella parte destra della figura seguente si riporta il modello corrispondente di grafo desiderato per diversi numeri di nodi).

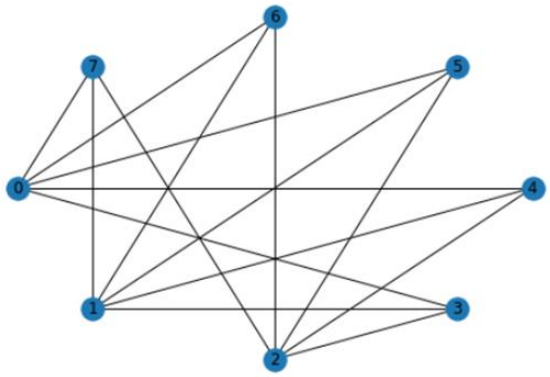
```
plt.clf()
barbell = nx.barbell_graph(5,0)
nx.draw_circular(barbell, with_labels = True)
plt.show()
```

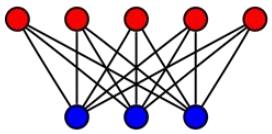




Visualizziamo, ora, alcuni grafici classici come il **grafo completo bipartito.** La funzione per ottenerlo è **complete_bipartite_graph**() a cui vanno passati come argomenti i nodi di ciascuna delle due partizioni (nella parte destra della figura seguente si riporta il modello corrispondente di grafo desiderato).

```
plt.clf()
K_3_5 = nx.complete_bipartite_graph(3,5)
nx.draw_shell(K_3_5, with_labels = True)
plt.show()
```





La «personalizzazione» del grafo con Networkx è abbastanza importante. Alcune *opzioni* possono essere inserite in un dizionario Python in cui la **chiave** è il nome dell'opzione a cui corrisponde un **valore** numerico o stringa come mostrato nel codice seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
options = {
    "font_size": 36,
    "node_size": 3000,
    "node_color": "white",
    "edgecolors": "black",
    "linewidths": 5,
    "width": 5,
}
```

Anche le **posizioni dei nodi** possono essere impostate in modo esplicito attraverso un dizionario Python dove:

- la **chiave** è il nodo;
- il **valore** è una tupla di due elementi con le coordinate 2D (x,y).

Nel codice seguente si impostano le posizioni di ciascun nodo del grafo e si creano gli archi tra tali nodi.

```
pos = {1: (0, 0), 2: (-1, 0.3), 3: (2, 0.17), 4: (4, 0.255), 5: (5, 0.03)}
```

```
g = nx.Graph()

g.add_edge(1, 2)
g.add_edge(2, 3)
g.add_edge(3, 4)
g.add_edge(1, 4)
g.add_edge(1, 5)
```

Infine, anche le **proprietà dell'area di visualizzazione** possono essere impostate in modo corretto a seguito dell'indicazione esplicita della posizione dei nodi come indicato nel codice seguente che permette di impostare i margini per gli assi in modo tale da non cancellare parte dei nodi.

```
ax = plt.gca()
ax.margins(0.20)
plt.axis("off")
plt.show()
```

Il codice completo per visualizzare il grafico con tutte le impostazioni analizzate risulterà essere, quindi:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

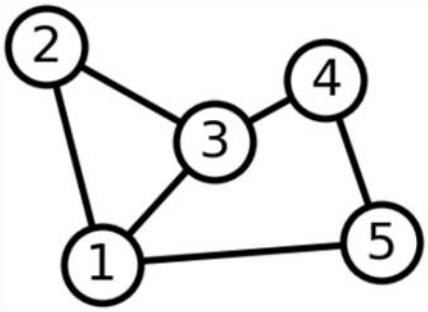
```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()
G.add_edge(1, 2)
G.add_edge(1, 3)
G.add_edge(1, 5)
G.add_edge(2, 3)
G.add_edge(3, 4)
G.add_edge(4, 5)

# posizioni dei noti esplicitate
pos = {1: (0, 0), 2: (-1, 0.3), \
       3: (2, 0.17), 4: (4, 0.255), 5: (5, 0.03)}

options = {
    "font_size": 36,
    "node_size": 3000,
    "node_color": "white",
    "edgecolors": "black",
    "linewidths": 5,
    "width": 5,
}
nx.draw_networkx(G, pos, **options)

# impostazione dei margini per gli assi in modo
#tale da non cancellare parte dei nodi
ax = plt.gca()
ax.margins(0.20)
plt.axis("off")
plt.show()
```



È possibile anche *impostare il grafico per colonne* (gruppi di nodi). Nell'esempio di un grafo orientato (nx.DiGraph()) è possibile, dopo aver aggiunto gli archi orientati, selezionare dei gruppi di nodi (liste) come nel caso successivo (left_nodes, middle_nodes e right_nodes).

```
G = nx.DiGraph([(0, 3), (1, 3), (2, 4), (3, 5), (3, 6), (4, 6), (5, 6)])

# gruppi di nodi per colonne
left_nodes = [0, 1, 2]
middle_nodes = [3, 4]
right_nodes = [5, 6]
```

Successivamente si imposta la coordinata x dei nodi sulla base della colonna di appartenenza (*left, middle, right*) come indicato di seguito.

```
# impostazione della coordinata x in base alla colonna
pos = {n: (0, i) for i, n in enumerate(left_nodes)}
pos.update({n: (1, i + 0.5) for i, n in enumerate(middle_nodes)})
pos.update({n: (2, i + 0.5) for i, n in enumerate(right_nodes)})
```

Si procede con il disegno della rete e l'impostazione dei margini dell'area del grafo come visto nel caso precedente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
nx.draw_networkx(G, pos, **options)

# impostazione dei margini per gli assi in modo
#tale da non cancellare parte dei nodi
ax = plt.gca()
ax.margins(0.20)
plt.axis("off")
plt.show()
```

Il codice completo per ottenere il grafo così come visualizzato a destra della figura seguente risulterà essere, quindi:

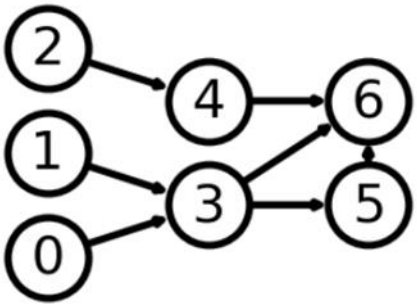
```
G = nx.DiGraph([(0, 3), (1, 3), (2, 4), (3, 5), (3, 6), (4, 6), (5, 6)])

# gruppi di nodi per colonne
left_nodes = [0, 1, 2]
middle_nodes = [3, 4]
right_nodes = [5, 6]

# impostazione della coordinata x in base alla colonna
pos = {n: (0, i) for i, n in enumerate(left_nodes)}
pos.update({n: (1, i + 0.5) for i, n in enumerate(middle_nodes)})
pos.update({n: (2, i + 0.5) for i, n in enumerate(right_nodes)})

nx.draw_networkx(G, pos, **options)

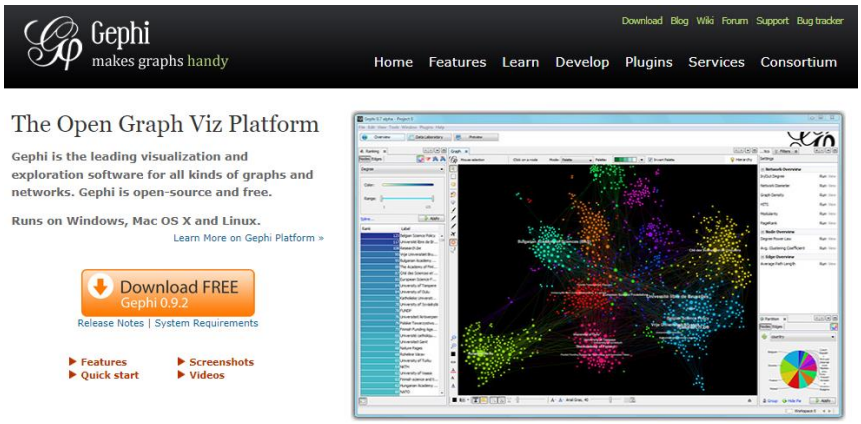
# impostazione dei margini per gli assi in modo
#tale da non cancellare parte dei nodi
ax = plt.gca()
ax.margins(0.20)
plt.axis("off")
plt.show()
```



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

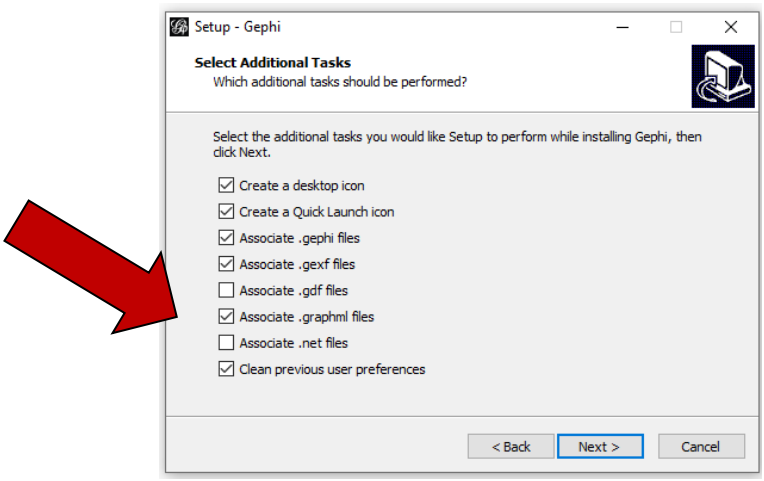
3. Visualizzazione dei grafi con Gephi

Gephi è un software *esterno a Python* per la **visualizzazione** e l'**esplorazione** di tutti i tipi di grafici e reti. Gephi è *open source e gratuito* ed è disponibile sul sito ufficiale: https://gephi.org/



Procediamo con il download cliccando su «Download FREE». Quando appare la seguente schermata, è necessario cliccare sul checkbox per associare anche i file con estensione **graphml.**

Questo formato sarà il *formato di esportazione* di grafi con Networkx.



Il codice Python per il salvataggio del grafo in un file per il *successivo editing* con **Gephi** è molto semplice. Nell'esempio riportato, si genera un semplice grafo e si procede con il salvataggio in un file con estensione ***graphml***.

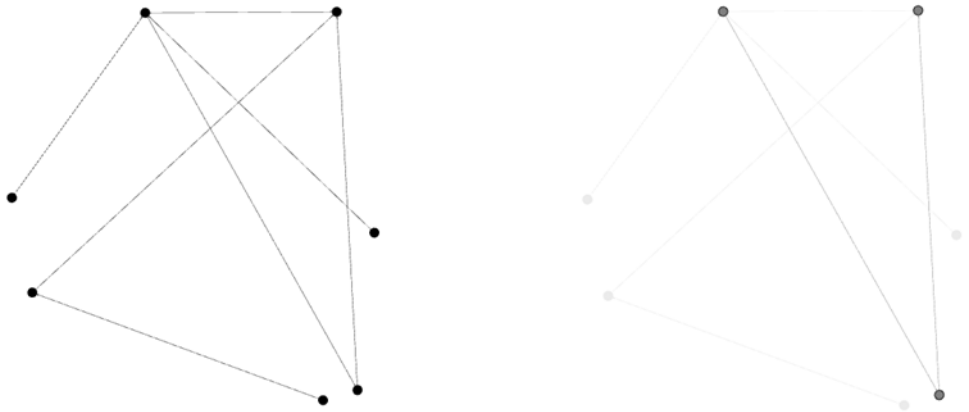
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import networkx as nx

G = nx.DiGraph([(0, 3), (1, 3), (2, 4), (3, 5), (3, 6), (4, 6), (5, 6)])

with open("esempio.graphml", "wb") as graph:
    nx.write_graphml(G, graph)
```

La *visualizzazione* di base del grafo salvato in Gephi è mostrata nella figura successiva (nella parte sinistra). Se si passa con il mouse su un nodo del grafo, vengono evidenziati solo i **vicini** del nodo stesso (parte destra). Gli altri nodi si vedono in trasparenza.



Una brevissima guida per l'editing dei grafi con Gephi è disponibile al seguente link: https://gephi.org/users/quick-start/

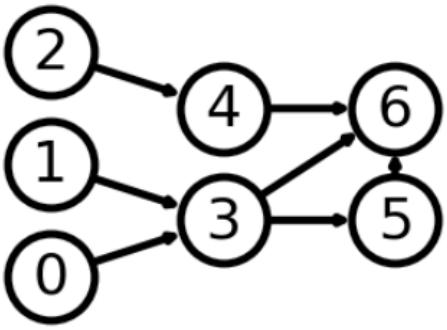
Altre risorse per approfondire Gephi sono disponibili al seguente link: https://gephi.org/users/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Analisi dei grafi

Per **analizzare ed esplorare** i *dati di una rete* con Networkx basta richiamare alcune funzioni e considerare i valori di alcuni attributi. Alcune proprietà come *ordine* e *dimensione* del grafo sono state già analizzate.

Un'altra proprietà interessante è il **grado medio dei nodi** ottenuto come la *dimensione del grafo* (numero degli archi) diviso l'*ordine del grafo* (numero dei nodi). Networkx mette a disposizione anche una funzione per il calcolo del **numero dei componenti fortemente e debolmente connessi**. Il codice seguente illustra diverse proprietà del grafo mostrato.




```
print("Numero nodi:\t",len(G))
print("Numero archi:\t",G.size())
print("Grado medio nodi:\t",float(G.size())/len(G))
print("Numero di componenti fortemente connessi:\t",nx.number_strongly_connected_components(G))
print("Numero di componenti debolmente connessi:\t",nx.number_weakly_connected_components(G))
```

```
Numero nodi:     7
Numero archi:    7
Grado medio nodi:        1.0
Numero di componenti fortemente connessi:        7
Numero di componenti debolmente connessi:        1
```

Per il calcolo delle **componenti connesse** è necessario un **grafo non orientato**. Per ottenere il grafo non orientato a partire da un grafo orientato è possibile utilizzare la funzione di trasformazione *to_undirected*() così come mostrato nel codice seguente. Dopo l'invocazione della funzione ***connected_components***() per il calcolo delle componenti connesse, è possibile iterare sulle componenti per conoscere i suoi elementi.

PEGASO
Università Telematica

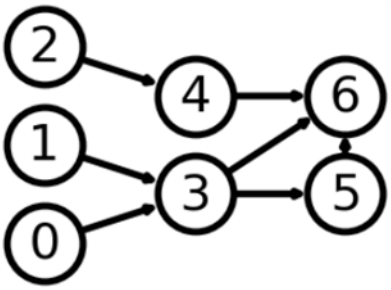
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
c = 0
o = nx.connected_components(nx.to_undirected(G))
for i in o:
    print(i)
    c+=1
print("\n",c,"componenti connesse")
```

```
{0, 1, 2, 3, 4, 5, 6}

 1 componenti connesse
```

Per i **grafi orientati**, invece, è possibile calcolare il **grado** (*entrante e uscente*) dei nodi e graficarne la distribuzione. Si riporta il codice per il calcolo del **grado** *entrante* dei nodi e la visualizzazione dell'istogramma relativo.

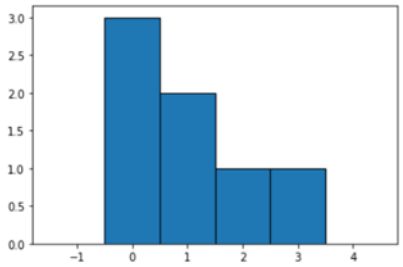



```
import numpy as np

l = list()
in_degrees= G.in_degree()  # tupla (nodo,grado entrante)
print("in_degrees:\t", in_degrees)
for e in in_degrees:
    l.append(e[1])
print(l)
plt.hist(l, bins=np.arange(-2,5)+0.5, ec="k")

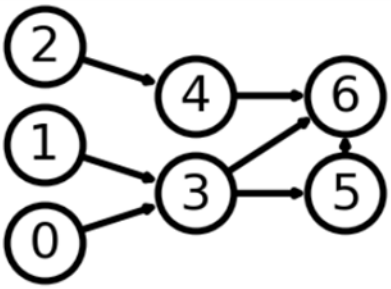
plt.show()
```

```
in_degrees:     [(0, 0), (3, 2), (1, 0), (2, 0), (4, 1), (5, 1), (6, 3)]
[0, 2, 0, 0, 1, 1, 3]
```



Si riporta, ora, il codice per il calcolo del **grado** *uscente* dei nodi e la visualizzazione dell'istogramma relativo.

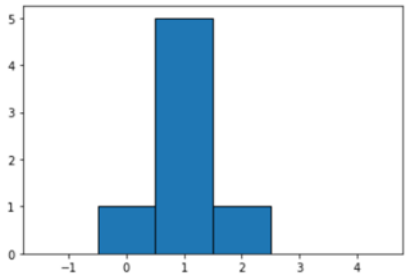
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).




```
l_out = list()
out_degrees= G.out_degree()  # tupla (nodo,grado entrante)
print("out_degrees:\t", out_degrees)
for e in out_degrees:
    l_out.append(e[1])
print(l_out)
plt.hist(l_out, bins=np.arange(-2,5)+0.5, ec="k")

plt.show()
```

```
out_degrees:     [(0, 1), (3, 2), (1, 1), (2, 1), (4, 1), (5, 1), (6, 0)]
[1, 2, 1, 1, 1, 1, 0]
```



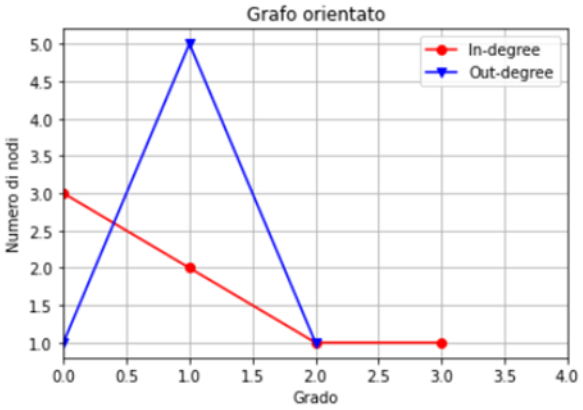
È possibile confrontare il grado entrante e il grado uscente dei nodi nel grafico della **distribuzione** generato e visualizzato con il codice seguente.

```
plt.figure()
plt.grid(True)

in_values= sorted(set(l))
print("in_values:\t",in_values)
out_values= sorted(set(l_out))
print("out_values:\t",out_values)

in_hist= [l.count(x) for x in in_values]
out_hist= [l_out.count(x) for x in out_values]
plt.plot(in_values, in_hist, 'ro-') # in-degree
plt.plot(out_values, out_hist, 'bv-') # out-degree
plt.legend(['In-degree', 'Out-degree'])
plt.xlabel('Grado')
plt.ylabel('Numero di nodi')
plt.title('Grafo orientato')
plt.xlim([0, 4])
plt.show()
```

```
in_values:      [0, 1, 2, 3]
out_values:     [0, 1, 2]
```



Il **coefficiente di clustering locale** di un nodo A è definito come il *numero di archi nel vicinato di A* (escludendo gli archi direttamente connessi ad A) diviso per il *massimo numero possibile di archi.* In Networkx è possibile calcolare il **coefficiente di clustering**:

- riferito ad un *nodo;*
- riferito a *tutti i nodi;*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- *medio riferito al grafo.*

Per operate tale calcolo, però, è necessario un grafo **non orientato**. Il grafo precedente dovrà, quindi, essere trasformato temporaneamente da orientato a non orientato.

Il codice seguente permette di calcolare il coefficiente di clustering di un nodo, il coefficiente di clustering per tutti i nodi e il coefficiente di clustering medio.

```
G_ud = G.to_undirected()

# coefficiente di clustering del nodo 0
print("Coefficiente di clustering del nodo 0:\t",nx.clustering(G_ud, 0))

# coefficiente di clustering di tutti i nodi (in un dizionario)
clust_coefficients= nx.clustering(G_ud)
print("\nCoefficiente di clustering dei nodi")
for i in clust_coefficients:
    print("Nodo:\t",i,"\tcoefficiente di clustering:\t",\
          "{:3.4f}".format(clust_coefficients[i]))


# coefficiente di clustering medio
avg_clust= sum(clust_coefficients.values()) / len(clust_coefficients)
print("\nCoefficiente di clustering medio:\t", "{:3.4f}".format(avg_clust))

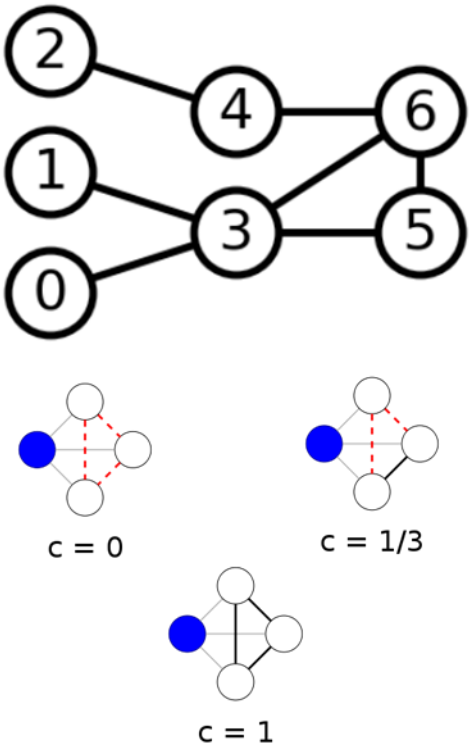
# metodo built-in direttamente
print("\nCoefficiente di clustering medio:\t",\
      "{:3.4f}".format(nx.average_clustering(G_ud)))
```

```
Coefficiente di clustering del nodo 0:   0

Coefficiente di clustering dei nodi
Nodo:    0      coefficiente di clustering:      0.0000
Nodo:    3      coefficiente di clustering:      0.1667
Nodo:    1      coefficiente di clustering:      0.0000
Nodo:    2      coefficiente di clustering:      0.0000
Nodo:    4      coefficiente di clustering:      0.0000
Nodo:    5      coefficiente di clustering:      1.0000
Nodo:    6      coefficiente di clustering:      0.3333

Coefficiente di clustering medio:        0.2143

Coefficiente di clustering medio:        0.2143
```



L'analisi dei grafi può proseguire con la ricerca dei **nodi con centralità maggiore.** I seguenti metodi, a cui viene passato come argomento il grafo, restituiscono (in un dizionario) le **misure di centralità** di *ciascun nodo*:

- *degree_centrality*() per la centralità locale;
- *in_degree_centrality*() per la centralità locale entrante;
- *out_degree_centrality*() per la centralità locale uscente;
- *closeness_centrality*() per la centralità globale;
- *betweenness_centrality*() per la centralità globale dei flussi;
- *eigenvector_centrality*() per la centralità ad autovettore.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tutti i metodi visti per il calcolo delle centralità restituiscono un **dizionario.** Per migliorare la «visibilità» dei nodi con più alti valori di centralità definiamo la funzione **get_top_keys**(). Questa funzione è utile sia per grafi con alto numero di nodi, sia per grafi con numero di nodi più limitato.

Grazie a tale funzione a cui passeremo il dizionario e un *parametro di soglia K*, le *migliori K chiavi* (corrispondenti a K nodi) verranno restituite in base ai **valori di centralità**. Si riporta di seguito il codice della funzione **get_top_keys**().

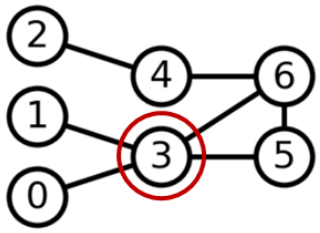
```
def get_top_keys(d, top):
    items = sorted(d.items(),reverse=True, key=lambda x: x[1])
    #items.sort(reverse=True, key=lambda x: x[1])
    return map(lambda x: x[0], items[:top])
```

Procediamo con l'analisi della **centralità locale** (*degree centrality*) che ricordiamo essere una misura relativa al numero di nodi adiacenti. Il codice seguente, dopo aver richiamato la funzione di calcolo della centralità, mostra il valore della centralità locale per ciascun nodo e, poi, per i tre nodi più influenti.

```
print("Degree_centrality")
d = nx.degree_centrality(G_ud)
for n in d:
    print("Nodo:\t",n,"\tdegree centrality:\t","{:3.4f}".format(d[n]))
print()
print("Nodi a più alta centralità locale")
best = get_top_keys(d,3)
for i in best:
    print(i,"{:3.4f}".format(d[i]))
```

```
Degree_centrality
Nodo:    0      degree centrality:      0.1667
Nodo:    3      degree centrality:      0.6667
Nodo:    1      degree centrality:      0.1667
Nodo:    2      degree centrality:      0.1667
Nodo:    4      degree centrality:      0.3333
Nodo:    5      degree centrality:      0.3333
Nodo:    6      degree centrality:      0.5000

Nodi a più alta centralità locale
3 0.6667
6 0.5000
4 0.3333
```



Procediamo con l'analisi della **centralità locale entrante** (*in-degree centrality*) per grafi orientati. Il codice seguente, dopo aver richiamato la funzione di calcolo della centralità, mostra il valore della centralità locale entrante per ciascun nodo e, poi, per i tre nodi più influenti.

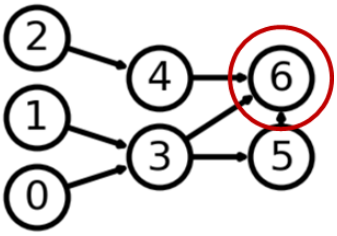
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("In-degree_centrality")
d = nx.in_degree_centrality(G)
for n in d:
    print("Nodo:\t",n,"\tdegree centrality:\t","{:3.4f}".format(d[n]))
print()
print("Nodi a più alta centralità locale entrante")
best = get_top_keys(d,3)
for i in best:
    print(i,"{:3.4f}".format(d[i]))
```

```
In-degree_centrality
Nodo:    0      degree centrality:      0.0000
Nodo:    3      degree centrality:      0.3333
Nodo:    1      degree centrality:      0.0000
Nodo:    2      degree centrality:      0.0000
Nodo:    4      degree centrality:      0.1667
Nodo:    5      degree centrality:      0.1667
Nodo:    6      degree centrality:      0.5000

Nodi a più alta centralità locale entrante
6 0.5000
3 0.3333
4 0.1667
```

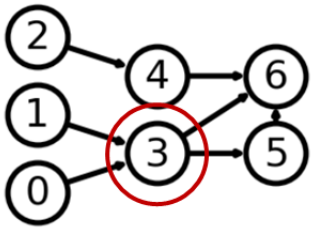


Procediamo con l'analisi della **centralità locale uscente** (*out-degree centrality*) per grafi orientati. Il codice seguente, dopo aver richiamato la funzione di calcolo della centralità, mostra il valore della centralità locale uscente per ciascun nodo e, poi, per i tre nodi più influenti.

```
print("Out-degree_centrality")
d = nx.out_degree_centrality(G)
for n in d:
    print("Nodo:\t",n,"\tdegree centrality:\t","{:3.4f}".format(d[n]))
print()
print("Nodi a più alta centralità locale uscente")
best = get_top_keys(d,3)
for i in best:
    print(i,"{:3.4f}".format(d[i]))
```

```
Out-degree_centrality
Nodo:    0      degree centrality:      0.1667
Nodo:    3      degree centrality:      0.3333
Nodo:    1      degree centrality:      0.1667
Nodo:    2      degree centrality:      0.1667
Nodo:    4      degree centrality:      0.1667
Nodo:    5      degree centrality:      0.1667
Nodo:    6      degree centrality:      0.0000

Nodi a più alta centralità locale uscente
3 0.3333
0 0.1667
1 0.1667
```



Procediamo con l'analisi della **centralità globale** (*closeness centrality*) che ricordiamo essere una misura della facilità di accesso di un nodo agli altri nodi. Il codice seguente, dopo aver richiamato la funzione di calcolo della centralità in un grafo non orientato, mostra il valore della centralità globale per ciascun nodo e, poi, per i tre nodi più influenti.

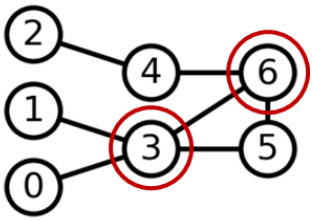
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("Closeness_centrality")
d = nx.closeness_centrality(G_ud)
for n in d:
    print("Nodo:\t",n,"\tcloseness centrality:\t","{:3.4f}".format(d[n]))
print()
print("Nodi a più alta centralità globale")
best = get_top_keys(d,3)
for i in best:
    print(i,"{:3.4f}".format(d[i]))
```

```
Closeness_centrality
Nodo:    0      closeness centrality:    0.4286
Nodo:    3      closeness centrality:    0.6667
Nodo:    1      closeness centrality:    0.4286
Nodo:    2      closeness centrality:    0.3529
Nodo:    4      closeness centrality:    0.5000
Nodo:    5      closeness centrality:    0.5455
Nodo:    6      closeness centrality:    0.6667

Nodi a più alta centralità globale
3 0.6667
6 0.6667
5 0.5455
```

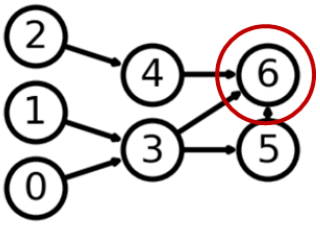


Ripetiamo l'analisi precedente, ma questa volta sul grafo orientato. I risultati sono riportati di seguito.

```
print("Closeness_centrality")
d = nx.closeness_centrality(G)
for n in d:
    print("Nodo:\t",n,"\tcloseness centrality:\t","{:3.4f}".format(d[n]))
print()
print("Nodi a più alta centralità globale")
best = get_top_keys(d,3)
for i in best:
    print(i,"{:3.4f}".format(d[i]))
```

```
Closeness_centrality
Nodo:    0      closeness centrality:    0.0000
Nodo:    3      closeness centrality:    0.3333
Nodo:    1      closeness centrality:    0.0000
Nodo:    2      closeness centrality:    0.0000
Nodo:    4      closeness centrality:    0.1667
Nodo:    5      closeness centrality:    0.3000
Nodo:    6      closeness centrality:    0.6667

Nodi a più alta centralità globale
6 0.6667
3 0.3333
5 0.3000
```



Procediamo con l'analisi della **centralità ad autovettore** (*eigenvector centrality*) che ricordiamo essere una misura della connettività con i vicini a livello globale (influenza di un nodo sulla rete). Il codice seguente, dopo aver richiamato la funzione di calcolo della centralità in un grafo non orientato, mostra il valore della centralità ad autovettore per ciascun nodo e, poi, per i tre nodi più influenti.

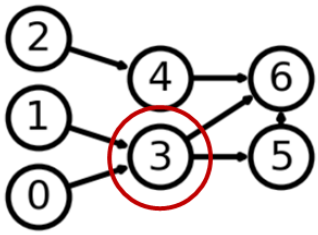
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("Eigenvector_centrality")
d = nx.eigenvector_centrality(G_ud)
for n in d:
    print("Nodo:\t",n,"\teigenvector centrality:\t","{:3.4f}".format(d[n]))
print()
print("Nodi a più alta centralità ad autovettore")
best = get_top_keys(d,3)
for i in best:
    print(i,"{:3.4f}".format(d[i]))
```

```
Eigenvector_centrality
Nodo:    0      eigenvector centrality:  0.2369
Nodo:    3      eigenvector centrality:  0.5846
Nodo:    1      eigenvector centrality:  0.2369
Nodo:    2      eigenvector centrality:  0.1023
Nodo:    4      eigenvector centrality:  0.2525
Nodo:    5      eigenvector centrality:  0.4479
Nodo:    6      eigenvector centrality:  0.5207

Nodi a più alta centralità ad autovettore
3 0.5846
6 0.5207
5 0.4479
```

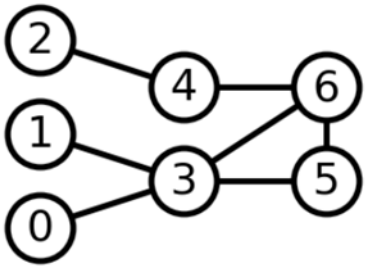


Per le **cricche**, Networkx mette a disposizione la funzione ***find_cliques***() che individua le cricche massimali. In particolare, *find_cliques*() non è implementata per i grafi orientati (i digrafi, infatti, devono essere prima convertiti in grafi non orientati) e restituisce un **generatore di liste di nodi**. Nel codice seguente, si richiama la funzione *find_cliques*() e si itera sul risultato per conoscere tutte le cricche massimali (in questo caso 5) con l'indicazione dei nodi che ne fanno parte.

```
cont = 0
cricche = nx.find_cliques(nx.to_undirected(G))
for c in cricche:
    cont+=1
    print("Cricca massimale n.",cont,"\t",c)
print("\n",cont,"cricche massimali")
```

```
Cricca massimale n. 1    [2, 4]
Cricca massimale n. 2    [3, 0]
Cricca massimale n. 3    [3, 1]
Cricca massimale n. 4    [3, 5, 6]
Cricca massimale n. 5    [4, 6]

 5 cricche massimali
```



Per ottenere, invece, la **cricca massima**, è possibile trovare la o le più grandi *cricche massimali* andando ad elaborare l'elenco delle cricche massimali nel seguente modo:

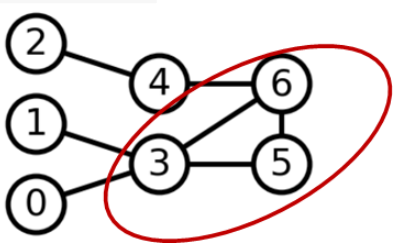
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import pandas as pd
l = list(nx.find_cliques(nx.to_undirected(G)))
cricche_massimali = pd.Series(l)
sb = cricche_massimali.map(lambda x: len(x))
sbmax = cricche_massimali.map(lambda x: len(x)==sb.max())
cricca_massima = cricche_massimali[sbmax]
print("Numero di cricche massime: ",len(cricca_massima))
print()
for c in cricca_massima:
    print(c)
```

```
Numero di cricche massime:  1

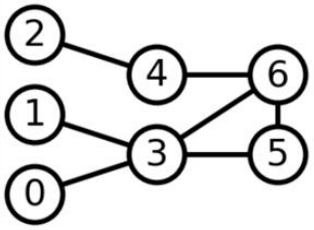
[3, 5, 6]
```



Per conoscere i **nodi isolati** (non connessi), Networkx mette a disposizione la funzione **isolates**(). Nell'esempio in questione, i nodi sono tutti connessi.

```
cont = 0
nodi_non_connessi = nx.isolates(G)
for i in nodi_non_connessi:
    cont+=1
    print("Nodo non connesso n.",cont,"\t",i)
print("\n",cont,"nodi non connessi")
```

```
 0 nodi non connessi
```



Il modulo per il rilevamento delle **comunità**, ***community***, non fa parte di Anaconda. Deve, quindi, essere installato separatamente attraverso i seguenti passi:

- avviare come Amministratore del file **cmd.exe** (click destro mouse ed "*Esegui come Amministratore*") solitamente nella cartella C:\Windows\System32;
- avviare l'Anaconda Prompt digitando il seguente comando:

C:\ProgramData\Anaconda3\Scripts\activate.bat

C:\ProgramData\Anaconda3

- digitare il seguente comando:

pip install community

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- al termine dell'installazione precedente digitare il seguente comando:

conda install –c anaconda python-louvain

Sarà quindi necessario importare la libreria community nel seguente modo (l'alias è arbitrario):

import community as com

La funzione **best_partition()** di community impiega il metodo di Louvain e restituisce una **partizione comunitaria**, cioè un dizionario di etichette numeriche di comunità indicizzati in base alle etichette dei nodi. Nel codice seguente si riporta sia il risultato di best_partition(), sia l'elaborazione dello stesso per distinguere correttamente il nodo e l'etichetta di comunità (entrambi numerici).

```
import community as com

partition = com.best_partition(nx.to_undirected(G))
print(partition)
nodi = partition.keys()
print()
for n in nodi:
    print('Il nodo:',n,"si trova nella partizione:",partition[n])
```

```
{0: 0, 3: 0, 1: 0, 2: 1, 4: 1, 5: 2, 6: 2}

Il nodo: 0 si trova nella partizione: 0
Il nodo: 3 si trova nella partizione: 0
Il nodo: 1 si trova nella partizione: 0
Il nodo: 2 si trova nella partizione: 1
Il nodo: 4 si trova nella partizione: 1
Il nodo: 5 si trova nella partizione: 2
Il nodo: 6 si trova nella partizione: 2
```

Ne segue che la partizione più grande è la numero 0 che contiene i nodi [0, 3, 1].

La funzione **modularity**() di *community* restituisce la *modularità* della partizione. Se la *modularità* è troppo bassa (<<0,5), la rete non presenta una **netta struttura comunitaria** (o almeno una struttura comunitaria sulla quale contare). Nel caso in oggetto la modularità è bassa e pari a 0.21.

```
print(com.modularity(partition, nx.to_undirected(G)))
```

```
0.21428571428571427
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.
- Department of Computer Science and Technology, University of Cambridge - https://www.cst.cam.ac.uk/
- Networkx - https://networkx.org/
- Gephi-The Open Graph Viz Platform - https://gephi.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).